LANGUAGE DESIGN / RESEARCH SYNTHESIS

Mathematical Ideas, Checked Evidence

A unified evaluation of nine proposals for a more natural proof language

With a source review of automatic term synthesis and proof tactic suggestions in Leant

---

Prepared for Vladimir Reshetnikov

Codex, with independent reading and review passes

5 September 2026 (Pacific time)

Corpus: Contour, Loom, MathStep, Mathematical Intent / MPL, Mosaic, Motive, Outline, Reason, and Spine. This is a critical design synthesis and static implementation review. Proposed syntax and architecture are specifications; they are not a delivered language implementation or a measured usability result.

The decision in brief

The nine reports converge on a coherent research direction: a Lean-hosted language whose visible units are mathematical claims and transformations, whose omitted details become scoped proof obligations, and whose accepted work is retained as replayable evidence. Their agreement is much stronger than their different names suggest. It supports building one experimental infrastructure with competing interaction policies, rather than nine separate languages.

The central opportunity is to make the boundary between a mathematical idea and its formal realization explicit. "Reindex this finite sum", "differentiate this identity on a neighborhood", and "use symmetry to reduce to this case" can be useful language operations when they select precise theorem-backed contracts. Changing punctuation around tactics will not by itself remove the cost of choosing representations, discovering lemmas, or maintaining domain libraries.

Six shared commitments are directly supported by all nine reports: a Lean host; claim-centered proof structure; contextual obligations and method contracts; fixed mathematical meaning before proof search; frozen evidence and replay; and evaluation against equally equipped Lean, including abstraction costs. Appendix A gives a report-by-report section crosswalk. These are nine documents supporting six design propositions, not nine independent experiments demonstrating their effectiveness.

The main unresolved choices concern how much reconstruction happens automatically, how strongly the system enforces the author's stated method, when representation and witness choices require attention, and how much evidence readers should see. This report recommends a small typed protocol and two authoring frontends, an ordinary Lean frontend and a mathematical-step frontend, sharing the same methods, search services, and evidence format. That experiment can determine which gains come from the language itself.

Leant provides substantial implemented work worth reusing: structural candidate synthesis, exact candidate verification paths, behavioral assessment with explicit limits, provider and fragment handling, and goal-shaped advisory tactic search. The suggestion path also exposes concrete review targets around completion status, replay of the displayed suggestion, and isolation of speculative work. These are implementation-level distinctions; a common vocabulary of "verified" must not erase them. Section 8 evaluates both systems separately.

The next round should deliver one end-to-end finite-reindexing example, one local differentiation example, explicit negative neighbors, and a replayable artifact. A second synthesis sharpens this plan: try theorem-based method annotations, measure existing automation, expose statement conventions, and compare definition interfaces. Its corpus audit motivates investigation rather than a savings claim. Code and reader observations should guide more elaborate surface syntax.

Contents

1 Corpus, method, and strength of evidence

1.1 What was compared

The input is the nine TeX reports present under `docs/ideas` at Brainstorm revision `ddbc1d4`. We read their substantive text, their status and source notes, and relevant companions. Three independent reading passes each covered three reports; this synthesis compares the resulting analyses against source passages. The associated review notes and a machine-readable source register are included with the report. The original reports remain the primary sources for attribution [3, 4, 5, 6, 7, 8, 9, 10, 11].

Report	Distinctive emphasis used in this synthesis
Contour	Mathematical moves, detailed obligation ledgers, finite reindexing, and leakage-resistant evaluation.
Loom	Working contexts, reconstruction recipes, local calculus, and witness/specification coupling.
MathStep	An assertion layer above evidence, branch-sensitive constructions, and exhaustive rebuilding of induction cases.
MPL	Statement seals, distinction between proof and object synthesis, preservation of general hypotheses.
Mosaic	Three proof views, integral and counting examples, explicit levels of method conformance.
Motive	Domain packages, algebraic generality, argument provenance, and failure contracts.
Outline	Certified higher-level plans, alternative mathematical proofs, and maintenance-oriented evaluation.
Reason	Reconstruction contracts, explanatory limits, and a small executable evidence model.
Spine	Contextual holes, structural choices, stable references, and advanced analytic patterns.

These emphases are not exclusive feature claims. For example, many reports discuss views, witnesses, synthesis, and replay. Their titles and provisional language names do not establish incompatible technical designs.

A subsequent comparison incorporates selected ideas from Claude's *Nine Blueprints for a Mathematician's Proof Language over Lean*, at Brainstorm revision `70020efd267f43131b8ef386ea40577a9e67b7cd` [1]. This is a secondary synthesis of the same inputs, not a tenth independent design proposal or an extra vote in the convergence crosswalk. Its source, feature matrix, and heuristic corpus-audit artifacts are archived separately. We distinguish its reported measurements from this report's source checks and new recommendations.

1.2 Independent authorship is not independent validation

The reports were commissioned in parallel, but share the task, two repositories, much prior art, and a mathematical sample heavily concentrated in FabiusFunction. Their common architecture could reflect a productive insight, common background, or a shared blind spot. Authorship independence, as described by the user, does not justify probabilistic independence or a statistical confidence estimate.

We therefore count *explicit report support* only for carefully delimited features. The count says that a proposition appears in the documents. It does not measure implementation success, novelty, reader preference, or superiority to Lean. Absence of discussion would not count as opposition. Differences below are labeled as competing policies, differences in emphasis, or unresolved questions; they are not inflated into disagreements that the texts do not express.

1.3 Design evidence versus implementation evidence

All nine reports identify their proposed language syntax as unimplemented. Their paper soundness arguments explain a proof-construction discipline; they do not verify a parser, elaborator, compiler, or editor implementation. None supplies a comparative user study establishing reduced formalization effort.

Reason's companion is a real but narrow exception to the absence of executable artifacts: a Python model checks simply typed terms with products, scope, claim assembly, and origin-bound replay. Its 32 tests were rerun for this synthesis and passed. This supports those tested model behaviors only. It is neither a Lean kernel nor a dependent proof-language prototype [10, §18].

For the additional Leant review, the local checkout's HEAD was `3a40904be8a410d832d9ab6900b3d3e7b425eccb`, but its working tree contained modified and untracked synthesis code. The reviewed source snapshot is retained in `evidence/leant-reviewed-source.zip`, with file identities in the source register. Its Djex checkout was `63e7b2fa46d62c633e0b5a24a463e22b9e939df4`, different from the root commit's gitlink. Selected ProveIt observations use `7c4e3f109405b9805b35d27ae96bf09c7ee5f3d5`. These differ from the older ProveIt snapshot cited by most inputs; Reason records moving source access rather than an immutable commit. We do not silently treat these snapshots as identical. The source register records the actual inspected files and their correspondence to committed content. Both repositories were treated as read-only; their builds and test suites were not run. Source inspection, the small Python test result, and the PDF production checks are separate evidence.

2 What needs improving in Lean?

2.1 Four costs with different remedies

The reports' most useful diagnosis is more discriminating than "Lean is verbose". Their examples suggest four different sources of work.

1. **Mathematical work.** Choosing an invariant, strengthening an induction motive, finding the right normal form, or proving a regularity hypothesis is part of the argument. A language should help express and explore this work.
2. **Representation work.** Moving between inclusive intervals and ranges, natural and integer coefficients, functions and series, or bundled objects and their values often interrupts an otherwise clear argument. Reusable, checked adapters can make the intended view explicit once.
3. **Proof assembly.** Introducing binders, specializing quantified facts, projecting conjunctions, composing maps, and closing small arithmetic obligations are promising targets for bounded reconstruction and synthesis.
4. **Interaction and maintenance.** Finding a theorem name, understanding a failed elaboration, repairing a changed instance, and recovering the reason for a generated step require indexing, diagnostics, stable evidence, and good editing.

The remedy must match the cost. A new sentence form will not replace a missing library theorem. A powerful solver may reduce authoring keystrokes while producing an opaque argument. A domain library can make both the new language and ordinary Lean concise. Refactoring the mathematics itself may outperform either syntax.

2.2 Preserve what Lean already does well

Lean already separates extensible syntax and elaboration from a small checking core, and tactics produce terms for that core. A language hosted there can reuse dependent types, libraries, notation, and editor metadata [12]. Declarative proof text is also established prior art: Isar provides structured proof commands with explicitly stated intermediate propositions [14]. The proposed work should be evaluated as a new integration and interaction design, not as the invention of structured or readable formal proof.

Several reports deliberately retain compact existing Lean examples as controls: Contour discusses already appropriate public corollaries, and MPL examines a short scaling proof [3, §2.5][6, §9]. This matters. The goal is not to translate every successful Lean idiom into English. Existing `calc`, explicit theorem applications, and well-factored lemmas may already be the best representation of a proof.

Our ProveIt spot check gives a concrete control. The current `Fabius.binomial_inversion` is already an application of a triangular-kernel framework, and the module works for sequences in an additive commutative group. A new frontend that assumes a field would shorten some algebra while weakening the result. The language must preserve that generality or explicitly identify the restriction [15].

2.3 Naturalness has several audiences

Concision is not the same as naturalness. An analyst, a combinatorialist, and a category theorist use different conventions. An author searching for a proof needs alternatives and holes; a reader needs an argument they can follow; a maintainer needs dependencies and stable meanings. A single presentation need not satisfy all three at once.

The reports mainly study proofs of statements whose mathematical objects and library infrastructure already exist. They say less about *definition authoring*: introducing a quotient, a new structure, a diagram, a hierarchy of equivalences, or local notation. That is a serious boundary of the corpus. A better proof surface can be valuable without yet being a better language for all mathematics. The next evaluation should include at least one task requiring a new definition, not merely a proof with more convenient access to existing declarations.

3 Where the reports converge

3.1 Six shared commitments

Appendix A records explicit support in all nine reports for the following six commitments. The formulation is deliberately narrower than a claim that they agree on every detail.

1. **Retain Lean as the first checking host.** Build syntax, elaboration, method libraries, and interaction around its existing foundations. A new kernel is not needed to test the proposed

gains.

2. **Make assertions and mathematical steps visible.** The readable source should name the facts, constructions, reductions, and calculations that explain the proof; routine proof-state manipulation can be reconstructed.
3. **Give omissions exact contexts and contracts.** Every omitted premise remains an obligation. A domain method specifies how checked subproofs assemble into the advertised result, respecting dependent scope.
4. **Fix meaning before search.** Proof search must not silently select a different theorem, stronger hypothesis, algebraic structure, or data object to make the task easier. Material interpretation choices must remain inspectable.
5. **Discover once and retain evidence.** Rebuilding a completed proof should check accepted work in a specified environment, rather than depend on repeating the original open-ended search.
6. **Compare fairly.** Charge for methods and adapters; give the Lean baseline the same machinery; include failed examples and maintenance tasks.

This is a plausible common core. It is not a complete language specification: the syntax, granularity of a step, inference policy, evidence interchange, and reader interface remain choices.

3.2 Three important shared distinctions

Proof correctness versus statement fidelity. A proof can be valid for an unintended statement. The reports repeatedly distinguish checking the proof from reviewing resolved types, quantifiers, definitions, and instances. Current Lean validation guidance likewise distinguishes theorem validity from meaning and provides stronger checking procedures when needed [13].

A proof of a proposition versus a chosen object. A type such as $\mathbb{N} \to \mathbb{N}$ does not specify the successor function. Synthesis of an inverse, normalization, branch, or witness may change later mathematics. A property, a chosen representative, or a uniqueness theorem is needed in addition to type inhabitation. MPL and Spine make this a particularly explicit classification, but it recurs throughout the corpus [6, §12.4][11, §9.1].

Search failure versus refutation. Exhausting a budget, exhausting a structural search fragment, and proving the negation of a mathematical statement are different outcomes. A positive candidate can be checked against the original goal. A negative completeness claim also needs an adequate relationship between the searched fragment and that goal. These distinctions are directly relevant to Leant, not merely theoretical niceties.

3.3 The omissions are shared too

There is little empirical evidence on error recovery, learning time, multilingual prose, accessibility, mixed text/diagram reasoning, and library extension by ordinary mathematicians. The repeated choice of a Lean host is practical, but leaves open whether Lean's representation and elaboration conventions themselves cause costs that a frontend cannot remove. There is also no convincing evidence yet for how many domain methods can coexist before name resolution and inference become unpredictable. Consensus should guide a prototype, not close these questions.

4 What each proposal contributes

4.1 Contour: mathematical moves with an audit trail

Contour develops a particularly explicit path from a named move to a ledger of premises, a structured certificate, and a release artifact. Its finite reindexing example exposes both a bijection and summand equality; its series example keeps composition admissibility visible. It separates logical success, conformance to the stated move, and explanatory value [3, §§6,7,11,12].

Its strongest methodological contribution is the requirement to exclude the target theorem, downstream results, and answer-bearing caches when testing reconstruction [3, §15.2]. Otherwise a solver may rediscover the theorem being benchmarked by name. The principal implementation risk is that a richly specified ledger becomes a second formalization burden. The useful experiment is whether reusable contracts let an author operate mostly on mathematical objects while the ledger is generated.

4.2 Loom: contexts and recipes that retain local hypotheses

Loom's working contexts organize facts around a neighborhood, finite index domain, coefficient algebra, or induction scope. Recipes reconstruct conventional moves from these typed contexts. Its differentiation example emphasizes that the equality must hold near the point, and its witness syntax ties a chosen term to its exact specification [4, §§4,7,8].

This suggests a good boundary for automation: a context can index available facts, but cannot invent customary assumptions. The risk is an implicit domain package that becomes as hard to inspect as a global tactic configuration. Require lexical scope, an expansion view, and an explicit record of which recipe and contextual facts were used. Context indexing is an optimization; the actual dependent context remains authoritative. Its formal-series case also transports rational units into arbitrary commutative $\mathbb{Q}$-algebras without requiring a field or an injective algebra map [4, §5].

4.3 MathStep: structural completeness, not just algebraic brevity

MathStep distinguishes the layer of asserted mathematics from its elaborated evidence and includes a notable example beyond calculus and sums: reconstructing routine branches of an induction over derivations. Its proposed operation to rebuild the remaining constructors must cover the selected recursor exhaustively and must be invalidated if the datatype changes [5, §§8,10.2].

That is a strong test of generality. A mathematical-step language should help with structural arguments as well as symbolic normalization. The risk is a permissive "and similarly for the rest" command whose accepted cases become stale. A method must enumerate the constructors and expose the exact motive and recursive inputs. Test it by adding a constructor, changing an index, and omitting a recursive premise.

4.4 MPL: preserving the statement while simplifying the proof

Mathematical Intent, Formal Evidence, called MPL in its text, makes a statement seal and a semantic difference view central. Its examples explain how overstrong differentiability assumptions, a canonical replacement for an arbitrary solution, or an unnecessarily strong coefficient structure would alter the mathematics [6, §§6–7,12].

The strongest contribution is a boundary among proof-only, data-producing, and statement-resolving synthesis. This should shape the common protocol. A seal is a record of one resolved meaning, not a proof that the meaning is intended. The interface must make it feasible to review that record; merely showing a hash or a long elaborated term transfers the burden without solving the usability problem. MPL also sketches a concrete coefficient compiler whose shifted exponential-series rule treats support outside the valid index range separately. This is a reusable method candidate that preserves both arbitrary coefficient algebras and boundary cases [6, §8.4].

4.5 Mosaic: explicit choices about explanatory strength

Mosaic offers three synchronized representations: the mathematical narrative, an obligation structure, and checked evidence. Its integral flatness bound, symmetry argument, and square-root sum give complementary mathematical tests. Most usefully, it explicitly distinguishes a checkpoint profile from a method-constrained profile [7, §§4,6–9].

Both profiles can be logically sound. The first checks every stated claim and permits broad reconstruction; the second also constrains how a named operation is implemented. This is a real experimental choice rather than a superficial syntax preference. The stricter profile can preserve the argument's structure but may reject an illuminating alternative. Make such alternatives source edits with a changed explanation, not silent substitutions behind the old prose.

4.6 Motive: reusable domain packages and honest provenance

Motive treats conventions as domain packages containing checked moves, aliases, normalization policies, and explanation renderers. It gives useful warnings about transport direction, algebraic assumptions, and analytic moves whose premises constitute substantial mathematics [8, §8].

Its discussion of explanatory provenance is especially careful: forcing a lemma name to occur in a term does not show that the lemma mattered, and normalization can erase syntactic scaffolding. A realistic contract records the instantiated recipe and its obligations without claiming to establish semantic indispensability or to reconstruct a human mental process [8, §9.4]. This restraint should be adopted throughout the unified design. Motive's Richardson mass-one example is a useful control: idiomatic Lean already proves the normalization in two lines. Extra surface machinery needs a demonstrated benefit even when a domain offers many attractive new abstractions [8, §2.3].

4.7 Outline: proof plans can improve the argument itself

Outline builds larger certified plans, including symmetry reduction and a residual bound that brackets a root. Its square-root example changes the proof organization to double counting instead of merely suppressing the original successor-case arithmetic [9, §§6,7,9].

This is a valuable reminder that the shortest natural proof may require a better mathematical decomposition. The corresponding evaluation hazard is attributing that improvement to the language. Give the alternative argument and its new lemmas to the Lean baseline too. A higher-level plan earns its place by reusing a well-specified schema across examples, not by being a disguised theorem specialized to the demonstration. Its symmetry contract also transports the complete hypothesis context. A symmetric conclusion alone does not justify exchanging variables under asymmetric assumptions.

4.8 Reason: a small executed model with a clear boundary

Reason brings together reconstruction contracts, theorem-backed views, explicit outcome distinctions, and publication evidence. Its small Python companion makes some scope and evidence-boundary claims executable [10, §§10,12,18]. This is the corpus's clearest example of moving from architectural prose to a testable artifact while labeling its limited reach.

That model is a useful source of protocol tests. It does not demonstrate dependent substitution, Lean integration, mathematical methods, or the proposed reading experience. The next step should port the tested invariants into an actual host boundary and exercise them with a real mathematical transformation. Extending the toy checker indefinitely would be a detour from the shared Lean-hosted experiment.

4.9 Spine: difficult contexts and stable mathematical choices

Spine provides contextual-hole semantics, distinguishes proof/data/structural/ presentation holes, and emphasizes references that survive insertion of nearby text. Its examples range from finite convolutions to contraction-based uniqueness and coefficients of formal inverse germs [11, §§4–9].

Its insistence on certifying even unused source assertions is essential: a true final theorem does not validate every assertion printed above it. The analytic examples are good later stress tests, but too rich for the first implementation. Preserve their exact obligations as roadmap requirements while starting with a smaller finite move that can expose scope, representation, and replay end to end. The formal-root example supplies a strong later boundary test: over a general ring, a nonzero coefficient need not be a unit. Recursive division by it therefore needs the stronger algebraic evidence [11, §2.6].

5 Choices that remain open

5.1 Competing policies, with proposed defaults

Choice	Benefit and cost	Proposed first experiment
Checkpoint or method contract	Checkpoints allow flexible completion. Enforced methods preserve the stated plan but can constrain useful alternative proofs.	Support both; use method conformance for named transformations and open reconstruction for explicitly open claims. Compare identical tasks.
Implicit or explicit views	Inference removes repetitive casts. Too many possible views can change meaning or make resolution unpredictable.	One lexically scoped, explicitly selected view per nontrivial representation change; automatic evidence for its guards.
Controlled syntax or free prose	Controlled syntax has reliable scope and parsing. Free prose is familiar but leaves mathematical interpretation underdetermined.	Controlled stored source; prose can suggest an editable typed draft whose resolved interpretation is visible.

Choice	Benefit and cost	Proposed first experiment
Embedded or external frontend	An embedded frontend reuses Lean elaboration. An external frontend may offer independent tooling but duplicates semantics.	Lean expressions at the boundary, with a small versioned service protocol; defer an independent expression elaborator.
Search or explicit reasons	Broad search may close more goals. Explicit reasons improve predictability and explainability but require discovery support.	Bounded local search by default; accepted broader search records a concrete reason and its dependencies.
Frozen terms or scripts	Terms avoid repeating tactic search. Scripts remain readable and may migrate better, but replay their elaboration and automation.	Retain source, resolved evidence, and environment identity. Measure script replay and stronger evidence replay as separate modes.

5.2 Default inference must not redefine the task

Inference is not uniformly dangerous or uniformly helpful. Inferring an implicit proof that $i \leq n$ from interval membership normally changes no mathematical choice. Selecting a topology, branch of a root, interpretation of subtraction, or uniform witness may change the statement. The boundary should follow the effect of a choice, rather than whether its syntax happens to be implicit.

Lean may interleave elaboration and proof construction. "Fix meaning first" should therefore mean a defined export and search boundary, not a claim that all frontends can resolve every header in a single parser pass. An unresolved meaning-bearing parameter can be exposed or left pending. A solver must not fill it merely because one instantiation makes the proof easier.

5.3 References after edits: a concrete policy difference

Loom favors retaining resolved identities for natural references, while Outline gives lexical rules for expressions such as "this" and requires review when meaning changes [4, §3][9, §5]. Inserting an inequality above "by the previous inequality" distinguishes the policies: should the reference stay attached to its accepted node, or follow the newly preceding statement? Neither behavior is universally self-evident. Prefer stable stored identities for accepted references, and display a source-level change when the author asks for lexical retargeting. Test insertion, deletion, and copy/paste rather than settling this question with a parser convention alone.

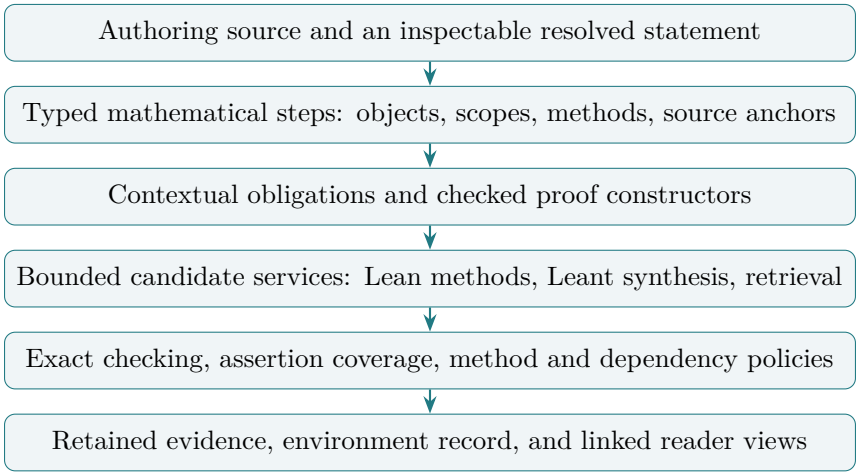
5.4 Bounded automation is an interaction contract

Budget the whole user operation, including candidate generation and checking. Multiple short probes can still cause a long pause. Require a total deadline, candidate limits, cancellation, distinct unsupported/exhausted/rejected outcomes, and stable origins that prevent stale results from changing a newer proof state.

Recommendation. Prototype inference, method conformance, and evidence display as policies of one shared protocol. Choose defaults after testing them.

6 A common architecture and its actual guarantees

6.1 From source to evidence



The diagram is a specification, not an implemented pipeline. Search can propose objects for several stages, but cannot grant them authority merely by returning success. The checking path retains the target fixed independently of the candidate.

6.2 A minimal obligation interface

For a fixed declaration environment $\mathcal{E}$, represent an obligation as

$$O = (id, \Gamma, T, \mathcal{D}, \mathcal{P}, origin),$$

where Γ is an ordered dependent local context, T is the expected type, $\mathcal{D}$ is the allowed dependency information, $\mathcal{P}$ is a search and acceptance policy, and the origin identifies the statement, source node, and environment. A successful candidate provides $\mathcal{E}; \Gamma \vdash t : T$, with the additional policy receipts kept distinct from that typing judgment.

The ordered context matters. In $\forall x\, \exists y\, R(x, y)$, the witness may depend on x. In $\exists y\, \forall x\, R(x, y)$, it may not. Closing a goal over its telescope before sending it to an external provider makes the dependency explicit. Restricting premises also requires retaining dependencies needed to type the remaining context; a list of permitted printed names is insufficient. Keep allowed local premises, allowed library declarations, and the transitive axiom policy as three distinct controls, as the secondary review usefully emphasizes [1, §5.1]. Two visually separate goals may still share a witness, type metavariable, or instance constraint. They can be solved independently only through closed typed interfaces or a transactional assignment discipline that commits compatible substitutions and restores failed alternatives. A proof-obligation DAG alone does not capture all elaboration constraints [4, §8.3][3, §9.6].

A service response should distinguish at least:

```
Candidate(exact_term, exact_origin)
Refinement(child_obligations, typed_combiner)
Unsupported(reason)
BudgetExhausted(budget, observations)
FragmentExhausted(fragment, completeness_evidence)
Rejected(candidate, diagnostic)
Checked(proof_receipt, policy_receipts)
```

An exact proof of a negation can be represented as checked evidence for that negation. It is not inferred from `BudgetExhausted`. A fragment result keeps its fragment and adequacy assumptions even when its internal search is complete.

6.3 What the repeated soundness arguments amount to

Once unresolved type and instance constraints permit such a checked interface, represent an unfinished proof as a typed dependent function

$$K : \Pi(o_1 : Q_1) \cdots (o_m : Q_m).\, T_\star,$$

where each Q_i closes over its authorized local variables and may depend on earlier obligations. The theorem type $T_\star$ is fixed and contains no free obligation parameters. These parameters are holes to solve, not axioms.

Proposition 1 (Conditional assembly)**.** *If K is well typed, and each p_i has the type Q_i after substitution of the preceding accepted arguments, then $Kp_1 \cdots p_m : T_\star$.*

Proof. Repeated dependent function application substitutes each checked argument into the remaining telescope. The required type for the next argument is exactly the substituted type in the hypothesis. After the final application, the remaining type is the fixed target. Ordered, well-scoped substitution prevents use of an unavailable variable; a dependency order prevents circular completion. □

This common argument is useful but modest. It is essentially the host type theory's composition property. The hard engineering work is preserving its hypotheses through parsing, elaboration, substitution, cache reuse, and export. Final rechecking protects against producer mistakes; it does not prove that a display renderer, search policy, or statement interpretation is faithful.

6.4 Four acceptance properties, not one green light

1. **Target fidelity:** the proof is checked against the intended resolved target in the specified environment. Equality of printed text is insufficient.
2. **Logical evidence:** the target has checked evidence under a disclosed transitive axiom policy, without unresolved holes or unauthorized assumptions.
3. **Document coverage:** every asserted source claim, including an unused intermediate claim, has evidence in its declared scope.
4. **Method conformance:** a method-constrained node was assembled using its declared contract, with its actual inputs and obligations recorded.

Human explanatory quality remains a fifth, empirical judgment. A valid proof of the final theorem could coexist with a false unused sentence in the document. Certifying every asserted formal claim fixes that particular problem. It still cannot prove that the argument is illuminating or that a cited lemma was indispensable.

The same separation applies to operational trust. A producer that emits terms can be logically untrusted, but executing its metaprograms in the checking process grants operational capabilities. A publication workflow should choose a checking boundary appropriate to its input; it should not describe shared-process execution as sandboxed simply because the output is later typechecked. Existing Lean tooling already provides several levels of proof validation [13].

6.5 What to freeze, and what can change

Retain the resolved target and relevant definitions, context and instance choices, source anchors, instantiated methods, accepted terms or a defined export form, dependency and axiom inventories, and the environment needed to replay them. Search logs are optional diagnostics; they are not the evidence itself.

A successful tactic script is useful but may rerun simplification, typeclass search, or theorem search. Stronger evidence replay has a different cost and compatibility boundary. Neither mode promises byte-for-byte stability across arbitrary Lean or library versions. Cache identity must include semantic inputs, and a cache hit must still have an acceptance policy. Hashes identify records; they do not establish truth or equivalence of changed declarations.

7 Shared mathematical tests that expose the design

The following derivations are mathematical explanations and proposed language tests. Their pseudocode has not been compiled as a new language. Each pairs a useful positive case with a nearby invalid or meaning-changing case.

7.1 Finite reindexing: make the map the visible choice

For natural numbers $j \le n$, put $m = n - j$. All sums in this example take values in $\mathbb{Z}$. The map $i \mapsto j + i$ is a bijection from $\{0, \dots, m\}$ to $\{j, \dots, n\}$. Binomial orthogonality reduces to

$$\sum_{k=j}^{n} (-1)^{n-k} \binom{n}{k} \binom{k}{j} = \binom{n}{j} \sum_{i=0}^{m} (-1)^{m-i} \binom{m}{i} = \binom{n}{j} [m = 0] = [n = j].$$

Here $[P]$ is one if P holds and zero otherwise. The identity $\binom{n}{j+i}\binom{j+i}{j} = \binom{n}{j}\binom{m}{i}$ uses $n = j + m$. The case $n < j$ must be handled separately by the empty interval. It is not covered by pretending that natural subtraction is integer subtraction.

```
case j <= n:
  put m := n - j
  reindex the sum by i |-> j + i, from [0..m] to [j..n]
  use the binomial product identity at each i
  conclude by the alternating binomial row identity
case n < j:
  conclude because the interval is empty
```

A reindexing contract requires membership, injectivity, coverage, and equality of corresponding summands; it retains factor order unless a commutativity theorem permits changing it. Bounds can discharge natural-subtraction identities locally. The current ProveIt source makes these ingredients visible through interval/range conversion, `Nat.choose_mul`, arithmetic, and cast transport. Its final transform theorem already reuses a general kernel calculus [15].

Negative neighbor. Replace $[0..m]$ with $[0..m)$. The map now loses the upper endpoint. The method must identify failed coverage, not ask a broad solver to prove the original result by another route while retaining "reindex" as the explanation. Also test an infinite sum: the finite contract grants no right to rearrange it.

Evaluation implication. This is an excellent first vertical slice because it combines a meaningful explicit choice with routine guards. It is not evidence of a language gain unless ordinary Lean receives the same reindexing API.

7.2 Local differentiation: the neighborhood is part of the idea

Let $W, \varphi, G_n, G_n' : \mathbb{R} \to \mathbb{R}$. Suppose $U \subseteq \mathbb{R}$ is open, W has derivative $\varphi(W(x))$ at every $x \in U$, and $G_0(W(x)) = W(x)$. Suppose, at the values $W(x)$ actually reached for $x \in U$, that G_n has derivative G_n' and

$$G_{n+1}(W(x)) = G_n'(W(x))\varphi(W(x)).$$

Induction gives $W^{(n)}(x) = G_n(W(x))$ on U. In the successor step the induction identity holds in a neighborhood of each $x \in U$, so derivative congruence and the chain rule give

$$W^{(n+1)}(x) = G_n'(W(x))W'(x) = G_{n+1}(W(x)).$$

Loom, MathStep, and MPL study this autonomous-derivative pattern; Reason's affine-orbit example requires the same neighborhood-equality distinction. The inspected `AutonomousODE.iteratedDeriv_eq_comp` creates an eventual equality using `hs.mem_nhds hx`, then applies derivative congruence and composition. Its hypotheses require differentiability only at the relevant range values, not everywhere [16].

```
induct n, retaining the identity on U
at x in U:
  differentiate the induction identity near x
  use the chain rule and the two recurrence hypotheses
```

Negative neighbor. The functions $f(t) = 0$ and $g(t) = t$ agree at zero but have different derivatives there. A pointwise equality cannot satisfy the neighborhood-equality premise. A diagnostic should request locality, rather than add global differentiability or claim the theorem is false.

Evaluation implication. Count preserved hypothesis generality as a correctness requirement. Compare the failure explanation for the pointwise variant and the amount of author input needed to express the local context.

7.3 Exact arithmetic: derive guards from a counting balance

For $n \in \mathbb{N}$, let $s = \lfloor \sqrt{n} \rfloor$ and $S(n) = \sum_{k=1}^{n} \lfloor \sqrt{k} \rfloor$. Count the finite pairs

$$A = \{(j,k) : 1 \le j \le s,\ j^2 \le k \le n\}.$$

For fixed k, there are $\lfloor \sqrt{k} \rfloor$ choices of j. For fixed j, there are $n - j^2 + 1$ choices of k. Since $j^2 \le n$, all these counts are nonnegative. Thus

$$S(n) = \sum_{j=1}^{s} (n + 1 - j^2), \qquad S(n) + Q = s(n+1), \quad Q = \sum_{j=1}^{s} j^2.$$

The standard square-sum identity $6Q = s(s+1)(2s+1)$ supplies both the exact division and, from the balance, the order needed for natural subtraction:

$$S(n) = s(n+1) - \frac{s(s+1)(2s+1)}{6}.$$

This remains valid at $n = 0$, when both index sets are empty.

Mosaic, Outline, and Reason use this family of examples to show how a better argument can make side conditions consequences of one central relation. The inspected ProveIt implementation instead proves a rational formula by successor cases, proves divisibility and a subtrahend bound, then transports back to naturals [17]. The counting proof is an alternative mathematical route, not a description of that source's implementation.

Negative neighbor. $\mathrm{cast}(2 - 3) = -1$ is false when the subtraction inside the cast is natural subtraction: the left side is zero. Similarly, casting natural $1/2$ does not produce the rational number $1/2$. The view must retain the order and divisibility requirements, or use a different explicitly stated operation.

Evaluation implication. Offer the same double-counting lemma and proof plan to both frontends. Compare the total development cost, not a new short proof with the old long proof while giving only the former a better theorem.

7.4 Object selection: two roots are not interchangeable proofs

For $a > 0$, the equation $y^2 = a$ has two real solutions. A solver asked for y with that property may return either. If later mathematics calls the chosen object the positive branch, the specification must also require $0 < y$, or the author must name the branch explicitly. A type-correct inhabitant of $\{y : \mathbb{R} \mid y^2 = a\}$ has not established positive-branch intent.

```
choose r satisfying r > 0 and r*r = a
  using the positive-root construction
retain r and its specification as one accepted choice
```

MathStep's algebraic-branch case, MPL's arbitrary-versus-canonical solution distinction, and Spine's structural holes illuminate the same issue at different scales. Existence in `Prop`, construction of a dependent pair in `Type`, and noncomputable choice also have different elimination and policy requirements. The word "choose" must not conceal which interface is being used.

Negative neighbor. A later optimization replaces r by $-r$ because both satisfy the equation. The retained positivity evidence and downstream target no longer apply. Rechecking the new object against the full contract must fail.

Evaluation implication. Test visible semantic changes, not just theorem closure. A helpful system preserves an accepted choice and explains alternatives without allowing search success to decide the meaning of subsequent text.

8 Review of Leant's implemented work

Evidence status: Static inspection of the recorded local working-tree source, documentation, and tests, including uncommitted changes. No Leant build, backend execution, or regression suite was run in this review. Findings below distinguish observed code paths from proposed runtime tests.

8.1 Two related systems with different jobs

Leant's `:synth` constructs terms for a requested type using structural synthesis and additional provider/assessment machinery. Its proving interface instead starts from a live Lean proof state and proposes tactic steps shaped by the current goal. These complement one another but are not one interchangeable service [18, 19].

Term synthesis is particularly relevant to reconstructing applications, introductions, projections, and other typed assembly once the mathematical premises are available. Tactic suggestions operate closer to the user's next editing action: an introduction, a case split, an induction, or a closing tactic. A future mathematical-step frontend should retain both capabilities and expose the distinction in its protocol.

8.2 Term synthesis: the valuable boundary is richer than "typecheck"

The implemented structural fragment is richer than simple implication composition: it represents products and sums, quantified sorts, proper-type applications, instance/provider constraints, and supported constructor families. Term-indexed dependencies and unsupported content can remain opaque. Djinn and Exference have different search and negative-evidence behavior; Exference is heuristic, while Djinn's negative conclusion is qualified by projection completeness, unsafe atoms, and budget policy. The default Djinn choice budget is not uniformly bounded, so an adapter must impose its own complete resource policy [25, 21].

The inspected implementation distinguishes rendered variants of one semantic candidate and counts accepted candidate groups toward a quota. Its verification scheduler is lazy: zero quota does not inspect candidate input, and successful quota completion need not force the remaining stream. This is useful for bounded interactive work. The `Verified` wrapper has a deliberately narrow contract: it records acceptance by a supplied callback, not a solver certificate, behavioral proof, or kernel proof [20]. The preparation and rendering pipeline retains the source/search relationship, provider bindings, and typed candidate origin. Post-verification selection uses scoped occurrence handles and checks permutations rather than trusting a new list of equal-looking strings. Ranking profiles include balanced, compact, diverse, and legacy choices; their structural costs help order candidates but do not prove minimality or explanatory quality [25, 26, 27].

Fragment translation and provider handling recognize that a structural search engine does not understand all of Lean. Hidden structure and unsupported forms affect what can be concluded from failure. The README qualifies complete-search claims by the translated, provider-free structural calculus and describes bounded provider exploration separately [21, 18].

The behavioral path addresses a different question from inhabitation: whether a candidate meets the intended specification in the assessed problem. This is essential when several terms share a type. A constant failure function can inhabit an option-binding type, and a state transformer can typecheck while threading the wrong state. Behavioral assessment, rejection, inconclusive outcomes, and actual proof evidence must retain their respective authorities rather than collapse into a single "good term" flag.

8.3 Two behavioral mechanisms, with different evidence

The established Length model reasons about a restricted language of finite list-spine lengths, scalar and pair domains, and linear-integer queries. Ranking normally retains candidates; filtering needs its own authority, with independently replayed negative evidence. Raw solver outcomes and finite positive samples do not automatically authorize rejection or establish a universal specification of the Lean function [28].

The current uncommitted work adds a separate query using an actual Lean proposition:

```
:synth choose : (forall A : Type, A -> A -> A)
  where choose Nat 11 29 = 29
```

This ASCII rendering illustrates the query structure. The implementation parses the full host terms, checks the predicate under the candidate binder with `autoImplicit false`, then binds each exact candidate and attempts the supplied proposition using `by decide`. A successful positive proof gives "satisfied"; a successful proof of its negation gives "falsified"; failure of both remains inconclusive. Only candidates passing type and predicate checks consume the success quota [29].

This is a useful extension beyond an external behavioral model, but its result means exactly the proposition asked. The example checks one input; it does not prove that the function always returns its second argument. A finite conjunction proves that conjunction. Infinite universal specifications may be undecidable for this method and remain inconclusive. The feature documentation explicitly records live six-operation synthesis acceptance as pending. Source and focused test code exist; this review supplies no new execution receipt, and the current verdict is not yet an archived independent specification-proof bundle.

Reuse judgment. Preserve Leant's candidate identity, origin, capability, budget, and outcome work. Adapt it through a narrow protocol that receives the exact original Lean obligation. Do not make a reduced structural representation the authority for what theorem was proved, and do not treat provider enumeration as complete reasoning about arbitrary dependent mathematics.

8.4 Tactic suggestions: useful structure already implemented

In `Main.hs`, `goalCandidates` selects plausible operations from printed goal and hypothesis shapes. Parsing is used to propose candidates; Lean's response then determines acceptance. The candidate list includes quick finishers, `exact?`, named introductions, decomposition, branching, induction, and simplification/search. `apply?` is intentionally excluded, with a documented backend failure rationale [19].

`suggestTactic` probes a persistent proof-state identifier without pushing accepted probes onto the user's proof stack. It retains up to three accepted, nonclosing candidates while looking for a closer, then tries composed finishers, including `simp_all` and an arithmetic continuation. Suggestions are

cached by proof-state identifier, and `:suggest` can reprint the result without repeating the search. This is a concrete improvement over selecting the first tactic that merely succeeds.

The existing `prove-suggest` transcript exercises conjunctions, undo, existential decomposition, list induction, and a recursive doubling example. It provides specific intended behaviors to preserve [24]. These source fixtures were read, not rerun; they are not fresh runtime receipts.

8.5 Three implementation boundaries to strengthen

Completion evidence. In the inspected suggestion path, `probeOnce` checks fatal responses, error-severity messages, and the existence of a successor proof state. Closing classification then uses a reduction in the number of visible goals, with a special textual check for a remaining-subgoals comment. It does not consult a structured `proofStatus` field there. `cmdQed` rejects visible goals but otherwise warns if the stored script contains the text `sorry` and proceeds to its save path. A warning is compatible with draft work; it is not a strict completed-proof gate [19].

There are safeguards worth preserving. Standalone `:qed` re-elaborates the assembled theorem source, so it can catch a malformed stored replacement before saving. When resuming a `sorry`, it instead prints replacement source for the enclosing declaration. Draft admission is intentional and does not demonstrate a kernel soundness defect. A distinct sealed-finish operation should apply the stronger target and axiom policy required by the proposed language.

The review concern is precise: a response with no visible goals but unresolved proof content must not be advertised as complete on goal count alone. A smaller goal list also denotes current-goal progress/closure, not necessarily completion of the entire proof. Define these statuses separately and preserve backend completion metadata. Test missing or malformed goal fields, admitted subproofs, unresolved metavariables, and one closed goal with others remaining. These are proposed regression cases; this review has not demonstrated their live behavior. The separate status has concrete protocol support: locally cached REPL v1.3.18 source checks the root term, its metavariables, kernel acceptance, and admissions after the visible goals disappear, and reports `proofStatus` separately. That source is recorded with the review; it is not a claim that every Leant invocation uses that cached binary [30]. Likewise, the existing golden fixture intentionally calls a two-goals-to-one transition "closes the goal". Preserve that useful local outcome while withholding whole-proof completion.

The displayed artifact. The direct probe may succeed using a question-mark tactic, while `parseTryThis` extracts different text for display. The selected text is not independently replayed before the direct path can conclude and cache it. Chained `exact?` results likewise splice displayed text under conditions about message count and line count. Those conditions improve formatting, but do not prove that the exact displayed replacement has the same effect [19]. The actual tactic-application path also stores extracted replacement text beside the state produced by the original tactic. Its script and state histories therefore need the same exact-replay association, even before final publication.

Replay the exact proposed edit against the same immutable origin before describing that edit as verified. A diagnostic message is a candidate source, not an authority for equivalence with the command that emitted it. Test a tactic that succeeds but prints an invalid or differently scoped suggestion, and a multiline valid suggestion whose indentation must survive formatting.

Speculation and responsiveness. The per-probe heartbeat bound limits some Lean work, but `probeOnce` uses the session backend. Its error branch invokes an emergency exit if that backend fails. Persistent proof-state identifiers isolate logical state updates; they do not isolate process failure or

arbitrary effects of executed metaprograms. Per-candidate heartbeats are also not a total user-facing deadline [19, 23]. The current emergency path clears stale backend identifiers and preserves the script; isolation should retain that recovery behavior rather than replace it with an assumption that workers never fail.

Move risky or potentially expensive generated-text replay to a disposable worker with an exact origin and an overall budget. Return accepted source and evidence to the live session only after checking. Test timeout, cancellation, process failure, and a late result after undo. This is a reliability requirement for proactive help, not a claim that every existing candidate currently fails.

8.6 What Leant does not yet supply for the proposed language

Leant's chronological replay planner explicitly distinguishes reuse, suffix replay, and full replay after history replacement [22]. That is a good local session abstraction. It is not by itself a theorem-publication format with target identity, source-assertion coverage, method certificates, transitive axiom policy, and library-version compatibility.

Ordinary candidate checking also uses a textual goal in the current environment with `autoImplicit true`. This is convenient for a permissive REPL, but it is not the approved-expression and dependent-telescope interface proposed here. The inspected check rejects immediate errors and admissions; a separate publication layer must enforce the transitive axiom inventory and retain exact evidence [21, 19].

The language experiment therefore needs an additional layer: contextual mathematical obligations, named method contracts, evidence linked to source assertions, stable choices of data and structures, and a defined replay boundary. Reuse existing engines and their tested scheduling ideas rather than prematurely rewrite all synthesis in Lean. Keep a Lean-native adapter as an architectural option if duplicated representations prove to be the limiting cost.

Recommendation. First make the accepted-candidate boundary exact and testable, especially for displayed tactic edits. Then integrate term synthesis as one provider for typed obligations. This builds on Leant's strongest existing work and gives the new language an honest account of what each result establishes.

9 Additional ideas and refinements

These are extensions of the combined design, not claims of historical novelty. Some build directly on ideas already present in several reports; the additional contribution is a more specific contract or experiment.

9.1 A semantic change budget for automatic edits

Classify a proposed edit by its effect, then allow automation appropriate to that effect. Replacing a proof term for the same fixed proposition is different from changing a witness, selecting a new local structure, or strengthening a theorem. The reports already motivate these distinctions. A useful refinement is to expose them as an edit protocol with four outcomes:

```
EvidenceOnly: same resolved statement and accepted objects
PlanChange: same theorem, changed mathematical decomposition
ObjectChange: a retained witness or structure changes
StatementChange: binders, hypotheses, definitions, or conclusion change
```

The first category may be automatic within policy. The others produce a visible mathematical difference view. A stronger induction motive is typically a plan change; it need not alter the public theorem, but it changes the local hypotheses and should be presented as such. This avoids conflating all edits that preserve the final type with harmless proof plumbing.

9.2 Compare chosen objects through explicit observations

Full equality is sometimes stronger than downstream mathematics needs. Suppose later claims observe a witness $w : A$ only through functions $o_i : A \to B_i$. Define a scoped relation

$$w \sim_{\mathcal{O}} w' \quad \Longleftrightarrow \quad \bigwedge_{i=1}^{r} o_i(w) = o_i(w').$$

If every affected downstream construction is proved to respect this relation, then replacing a representative can be justified without pretending that the representatives are equal. This could help with quotients, choices of coordinates, normal forms, and constructions unique only up to a relevant equivalence.

The qualification is essential. Similar-looking uses do not establish that all dependencies respect the relation. A first implementation should require explicit transport lemmas and a closed set of registered observations. New observations invalidate the replacement certificate. Dependent result types may require a dependent transport interface, not merely equality of a few scalar projections. This is a later research direction; the safe initial policy is to retain the accepted witness unchanged.

9.3 Make visibility responsive to mathematical choices

An obligation ledger can overwhelm readers as effectively as a tactic trace. Default visibility should depend on *why* an obligation exists: show branch choices, induction motives, new regularity assumptions, and representation-changing guards; collapse routine proofs whose propositions are already apparent from the local context. A bound such as $i \leq n$ may be routine inside a bounded sum, while domination for a limit interchange is often the central argument.

This policy should be editable by author and reader, and generated from typed nodes rather than persuasive prose. Evaluate whether readers can locate a missing assumption quickly. Do not optimize only for the percentage of obligations hidden: concealing the decisive premise is a failure even if the page is shorter.

9.4 Turn negative neighbors into the method's specification

Every contributed method should arrive with a positive instance, a nearby failure, and an edit that changes its applicability. Finite reindexing gets an endpoint omission; differentiation gets a pointwise-only identity; symmetry gets asymmetric hypotheses; arithmetic transport gets failed divisibility; induction gets a new constructor. This develops the reports' negative-test proposals into a reusable package interface.

The expected result must include the missing proposition and the source location, not only "reject". A system that refuses everything is safe in a vacuous sense and useless. Positive neighboring cases measure whether the method is practical; negative cases measure whether it expresses the intended mathematics.

9.5 Build a bilingual proof document before a standalone language

Use the same typed step structure to render both a controlled mathematical source and an expanded Lean view. Allow an ordinary Lean proof block as an explicit leaf and let an author gradually replace repetitive patterns with reusable methods. The reports already support progressive adoption; the refinement is to make the two frontends the primary controlled experiment.

The Lean view need not be a perfect textual round trip through arbitrary Lean metaprograms. Define the subset with round-trip guarantees and preserve opaque escape blocks verbatim with their checked boundary. Source anchors should identify assertion nodes rather than depend on the phrase "the previous inequality". A formatter may use natural references only when it can recover that stable target.

9.6 Make the logical core of a method an ordinary theorem

The secondary synthesis makes a useful implementation proposal: represent many methods by ordinary Lean theorems with role annotations on their binders, instead of duplicating the theorem in a separate contract language [1, §7.1]. The theorem's dependent telescope supplies the logical premises; applying it assembles the proof. An annotation distinguishes author-selected inputs from premises that a specified routine profile may attempt. Composite moves can first be packaged as ordinary reusable lemmas, available to both frontends.

For a reindexing theorem, the index map is typically an author-selected input, while membership, injectivity, surjectivity, and summand correspondence become explicit obligations. Which of these is routine depends on the example; the annotation must not globally classify bijectivity as mathematical trivia. For local differentiation, a composite theorem can collect the neighborhood equality and derivative-existence premises already displayed in Section 7. Its statement remains the authoritative logical contract.

This reduces duplication, but does not make every method merely an attribute. Binder roles, permitted dependencies, normalization policy, resource limits, source locations, and explanation templates are operational or presentation contracts. They require their own versioning when the underlying theorem is unchanged. An ordinary theorem can specify a transformation without prescribing its search algorithm. Normalizers and structural plans also need interfaces for generated subgoals, binders, and transactional state. Register roles against resolved binder identities and validate dependencies; a positional list of flags is too fragile under theorem refactoring. Test the annotated-theorem design first, and add a richer method representation only for demonstrated requirements.

9.7 Reuse existing automation, while specifying what its record means

The secondary review usefully asks what existing Lean and Mathlib facilities can already supply before commissioning new machinery [1, §7.2]. This sharpens the common-library baseline: inventory the relevant theorem families, tactics, and information already available during elaboration, then

implement the missing persistence, policy, and user interaction around them. Existing proof terms already record formal evidence; the missing item is often a durable, reader-oriented account of the intermediate choices and obligations.

Mathlib's `says` combinator is a particularly relevant precedent. At the inspected revision, `X says Y` normally executes `Y`; discovery can be requested by writing `X says`. With verification enabled, including by default when the `CI` environment variable is present, the implementation instead executes `X` and compares its suggested syntax with `Y` after pretty-printing. That branch does not independently execute `Y`. These are different checks, and a suggestion consumer needs both in the appropriate modes. Moreover, `Y` remains an executable tactic sequence and may itself search. Thus `says` is a valuable discovery/replacement interface, not by itself a search-free proof-term archive, statement seal, or axiom-policy gate [2]. This distinction directly reinforces the Leant displayed-edit review in Section 8.

Reusable mechanism	Additional contract to test
Theorem application	Stable binder roles, exact target, scoped obligations, and an explanation tied to the instantiated theorem.
Proof-producing automation	Recorded premises and policy, predictable budgets, cancellation, and retained evidence with a defined replay mode.
Suggested replacement text	Independent replay of the exact displayed edit from the same state, with completion status kept separate.
Elaboration information	Persistent source-to-claim mapping, accounting for generated subgoals, and reviewable changes to meaning.

Calling the project an *accounting layer* is a productive first milestone, but an unmeasured upper bound on its eventual scope. The existence of a tactic does not show that a mathematician can select its inputs, interpret its failure, or recover its explanation cheaply. Conversely, when an ordinary library lemma already provides the desired interface, it should remain an ordinary library asset. The comparative experiment must determine the remaining language benefit.

9.8 Review statements through explicit semantic conventions

The secondary review develops the reports' statement views into two concrete features: diagnostics over the frozen expression and deterministic rendering back into mathematical language [1, §7.4]. Adopt these as aids to meaning review. The renderer should reveal resolved domains, quantifier dependencies, interval endpoints, selected structures, and totality conventions. For example, it can display natural subtraction as a truncated difference and show whether a chosen radius may depend on a previously bound parameter.

Separate two kinds of diagnostic. A *semantic notice* reports a resolved construction, such as the use of total division, without calling it an error. A *contract obligation* records an actual premise of a requested operation, such as the nonzero factor needed for cancellation. A theorem about division need not assume its denominator nonzero; that condition may be irrelevant, derived from context, or the subject of a case split. Likewise, a statement about `deriv` need not contain an explicit differentiability hypothesis. Token patterns cannot infer the author's intended quantifier dependence or decide whether a stronger algebraic structure was accidental.

Consequently, start with deterministic disclosure and a small configurable lint, with explanations and recorded dismissals. Measure false alarms as well as missed semantic changes. Rendering must

remain linked to the resolved expression; controlled English is another representation that needs validation, not an independent specification oracle. Test round trips for a defined subset and ask readers to distinguish nearby statements before using the view as an approval interface.

9.9 Treat definitions and representation interfaces as a parallel experiment

The secondary report's emphasis on definitions is valuable [1, §7.6]. Representing a bounded function by a subtype, choosing closed rather than half-open intervals, or exposing a total operator can impose recurring costs on later proofs. Our existing definition-authoring question should therefore become a concrete experiment alongside proof-block syntax. The original proposals recognize many such representation choices; the additional priority is to investigate their creation and library interface, not claim that none noticed the issue.

For one small mathematical object, compare two Lean representations using the same mathematical specification. Construct a definition package containing the resolved definition, explicit coercion and totality conventions, proved interface lemmas, and a documented simplification policy. Generated bridge lemmas must be proved, and their own development and maintenance must be charged to the package. Expose the same package through ordinary Lean and the proposed surface. Measure the downstream proof obligations, author effort, elaboration cost, and behavior after changing an instance or representation.

Include a composition test: transport through three views with dependent guards, then add a competing route. Record the route and its side conditions; require an explicit choice or a certified coherence policy when routes affect accepted objects or mathematical meaning. Directed transport theorems justify individual steps, but do not by themselves select a predictable route through a growing library. Definitions may offer large savings; the secondary report's claim that they are the larger lever remains a hypothesis this experiment can test.

10 How to evaluate the proposal without rewarding hidden work

10.1 Use corpus measurements to select methods, with calibrated claims

The secondary review contributes a useful change of scale: a lexical audit of 1,004 FabiusFunction files rather than only the reports' worked examples. Its archived output contains 90,637 classified tactic-like source entries. Of these, 28,727 (31.7%) are classified as assertions, and 29,303 (32.3%) fall into seven families the reviewer groups as candidates for reconstruction: explicit small obligations, casts, indices, representation changes, normalization, side conditions, and analysis. The analysis class accounts for 7.2% of the full tally and 2.9% of the recorded comparison sample [1, §3]. Our read-only rerun reproduced all 17 serialized full-subtree summary fields. The sample was not independently reproduced: its exact input-path manifest is missing from the shipped artifacts.

These are *reported lexical classifications*, not measured removable proof work. A tactic head can contain nested tactics, and a source-line classifier can miss continuations or conflate a deep theorem application with routine conversion. Assertion counts do not measure how much of the proof is covered by named claims. Classifying a line as analysis or representation work does not show that it can be reconstructed from the author's intended inputs. The audit's rounded proportions therefore supply neither a rigorous lower bound on removable work nor an estimated percentage saving. Absence of a textual `sorry` marker would not establish corpus completeness or successful Lean compilation.

The useful implication is a sampling priority: retain local analysis as a first class test, and look beyond short combinatorial examples when selecting routine profiles. Before implementing many methods, instrument a small, stratified set of compiled examples to record actual goals, scoped contexts, tactic outcomes, generated obligations, and elaboration cost. Lean elaboration information can support that collection; mapping it to mathematical roles still requires explicit definitions and manual calibration. Do not call the result free of heuristics.

For a proposed theorem adapter, freeze the pre-step goal and context, match its conclusion, and try its premises under a fixed routine profile and budget. Count successes, required author choices, unsupported cases, latency, and replay cost on held-out files. Use the same adapter in the Lean baseline. Export selected ledger entries through the closed typed interfaces of Section 6, after shared elaboration constraints have been settled or explicitly represented. Exclude the target theorem and downstream dependencies. Split tasks by theorem or module before harvesting subgoals, so neighboring obligations from one proof do not leak across training and evaluation. This turns a promising inventory into a test of actual demand.

10.2 A shared baseline and two distinct studies

Use at least three treatments: existing idiomatic Lean; refactored Lean with the same methods, adapters, retrieval, and synthesis providers; and the proposed mathematical-step frontend using that identical machinery. The second comparison isolates the incremental value of the language. The first measures the total package improvement available to the user.

Separate an *authoring study* from a *reading and repair study*. Authors need to build proofs and recover from failed attempts. Readers should explain the argument, identify dependencies, find an invalid move, and repair a small changed hypothesis. Randomize task order and counterbalance frontend order; record prior Lean and subject experience. Use a held-out mathematical family so demonstrations are not the entire evaluation set.

10.3 Measure a vector, not a single compression score

Dimension	Observable outcome
Author effort	Time to an accepted proof, editing actions, failed attempts, and explicit mathematical choices.
Reader understanding	Ability to state the key idea, locate a required hypothesis, and identify the effect of a proposed edit.
Maintenance	Repair time and affected nodes after a bound, definition, instance, theorem name, or datatype constructor changes.
Generality	Exact preservation of domains, quantifier dependencies, local hypotheses, and algebraic structure.
Interaction	Median and tail latency, cancellation behavior, stale result handling, and total checking work.
Evidence quality	Assertion coverage, exact displayed-edit replay, dependency disclosure, and defined replay success in a pinned environment.
Total cost	Shared infrastructure effort, per-example adapters, proof authoring, and subsequent maintenance.

Report source length and generated proof size as descriptive measurements, not proxies for all these outcomes. A one-line invocation of a purpose-built theorem can be shorter without explaining more or reducing total development effort.

Let L be the cost of a genuinely reusable method package, A_i the remaining authoring cost, and R_i maintenance cost for example i. Report

$$C(n) = L + \sum_{i=1}^{n} (A_i + R_i)$$

alongside each component. Amortization over held-out reuse matters; dividing an adapter cost over arbitrarily many hypothetical future proofs does not.

10.4 An initial benchmark with discriminating mutations

Task family	Positive task	Mutation or comparison
Finite algebra	Reindex an inclusive binomial sum.	Remove the upper endpoint; preserve the original additive-group generality.
Local analysis	Differentiate a family identity on an open set.	Replace local equality by equality at one point; compare diagnosis quality.
Counting	Derive the square-root sum using a finite relation.	Remove exact division evidence; compare both frontends with the same counting library.
Structural proof	Rebuild an induction over derivation constructors.	Add a constructor or alter an index; require regenerated obligations.
Object construction	Select a positive root with its specification.	Attempt substitution of the negative root after acceptance.
Compact control	Reuse a short scaling or transform theorem.	Measure whether the new frontend adds work without improving comprehension.
New domain	Define a small quotient-based or algebraic object.	Assess whether existing views and contracts support definition authoring.

The first two tasks are implementation gates; the others broaden the study after those work. Existing report examples are development material. Hold out variants and at least one domain when selecting methods. Remove the target theorem and downstream consequences from both retrieval and the allowed dependency closure; retain a record of the permitted environment. Inspect for hidden answer leakage through aliases, caches, examples, and imported packages.

10.5 Ablations that can change the decision

Disable one capability at a time: new surface syntax; contextual fact indexing; automatic guards; broad retrieval; method conformance; frozen evidence; and progressive disclosure. If a gain vanishes

when an adapter is removed but survives without new syntax, attribute it to the adapter. If the stricter profile improves reader diagnosis but harms authoring time, present that tradeoff directly.

Set go/no-go criteria before measurement. Exact targets and negative-case behavior are hard correctness requirements. Usability thresholds should be chosen after a pilot establishes realistic variance; invented percentages now would look precise without being informative. Publish failed tasks and repair costs as well as successful examples.

11 Questions for the next iteration

Each question below asks for an artifact or observation that would change a design decision. The next round should distribute competing implementations of the same contract, not request another set of unconstrained language manifestos.

Q1. What is the smallest useful mathematical step? Implement finite reindexing once as a theorem with binder-role annotations, once as a structured method with an obligation ledger. Show author input, errors, evidence, and the policies that cannot be expressed by the theorem type alone. Which additional layer reduces actual work?

Q2. When should a reason constrain the proof? Compare checkpoint and method-constrained modes on a valid alternative proof and a failed advertised method. Can readers tell which argument was checked? Should a changed method be an explicit source edit?

Q3. Which choices may reconstruction make automatically? Produce an edit classification for bounds, induction motives, root branches, instances, and changed hypotheses. Test a stable statement before and after each edit. Resolve disagreement with concrete examples.

Q4. What exactly does a verified suggestion guarantee? For Leant, specify and test exact displayed-source replay, current-goal versus whole-proof completion, incomplete response handling, timeout, and undo. Should the public receipt encode these distinctions?

Q5. How much domain infrastructure does concision require? Measure existing tactic and lemma coverage on a stratified sample before choosing the routine profile. Port the same reindexing and local-calculus contracts to a held-out family. Record new lemmas, matched-but-unsolved goals, and engineering time, including work needed to make Lean equally concise.

Q6. Can views remain predictable as libraries grow? Compose three views with dependent guards, then add a competing cast, coefficient convention, or structural instance. Measure whether interpretation changes, ambiguity is reported, or explicit scope preserves the prior meaning. Define local coherence requirements and show the recorded route.

Q7. What must an author actually inspect in a statement seal? Show concise renderings for changed quantifier order, a stronger domain, natural versus integer subtraction, and a different branch. Ask readers to identify the difference without reading a raw elaborated term. Compare semantic notices with policy diagnostics; count false alarms and missed changes.

Q8. Which retained artifact best supports maintenance? Replay a fixed proof, then rename a lemma, change a definition, and upgrade a dependency in an isolated experiment. Compare scripts, retained terms, and method-level evidence. Record repair effort and compatibility boundaries.

Q9. Does the language help introduce mathematics as well as prove it? Package one

definition involving a quotient, equivalence, or local structure, with proved bridge lemmas and explicit conventions. Compare two representations and their downstream costs through both frontends. Does the new surface help author the object, or is the benefit entirely in the shared library interface?

Q10. What should remain visible in a published argument? Compare a collapsed ledger with a view that exposes choices and substantial analytic premises. Measure explanation and error-localization performance, not subjective preference alone.

12 A practical next-round work plan

12.1 One shared contract, several bounded investigations

Agree on the obligation record, exact-target boundary, source-assertion identity, candidate outcomes, and a minimal method certificate. Assign one owner for this shared interface. Other work can proceed in parallel on separate contracts or frontends without creating incompatible foundational representations.

Lane	Concrete deliverable	Acceptance gate
Core/evidence	Lean-hosted scoped claims, one method combiner, exact checking, assertion coverage, and retained origin.	Reject circular/out-of-scope claims and mismatched targets; replay a completed example.
Leant adapter	Typed candidate exchange and exact displayed-tactic replay with bounded workers.	Distinct completion outcomes; cancellation and stale-origin tests; no speculative live-state mutation.
Finite methods	Reindexing plus guarded arithmetic transport in both frontends.	Positive example and endpoint/divisibility failures; same library available to Lean.
Analysis/UI	Local differentiation contract and linked source/obligation views.	Preserve local hypotheses; diagnose pointwise-only equality; reader can locate the neighborhood premise.
Evaluation	Counterbalanced pilot, held-out tasks, and cost register.	Calibrate corpus labels against actual goals; report authoring, comprehension, failures, and maintenance separately.

12.2 Milestones and stopping rules

Preparation: measure a bounded sample. Inventory existing mechanisms and instrument representative finite, analytic, and definition-heavy examples. Select a small routine profile from measured outcomes. A whole-corpus rebuild is not a prerequisite for the first end-to-end prototype.

Milestone A: one complete path. Author a short finite proof, inspect its obligations, check all assertions, export its evidence, and replay it without rerunning discovery. Do not expand the grammar until this path works.

Milestone B: a semantically difficult neighbor. Add local differentiation and a chosen data object. Demonstrate failure of the nearby invalid variants. This tests whether the design handles mathematical meaning, not only arithmetic.

Milestone C: comparative evidence. Run the shared-method frontend comparison and a repair exercise. If improvements come mainly from library work, keep that work and reconsider how much new syntax is justified. If the language helps readers but costs authors, offer it as a document view rather than force it as the only input form.

Large analytic plans, automatic extraction of reusable methods from successful proofs, broad natural-language parsing, and observation-relative witness replacement should wait until the smaller boundary has credible evidence. They remain research opportunities, not prerequisites for an initial useful tool.

13 Assessment and limits

The most defensible synthesis is a language of *explicit mathematical choices with reconstructible formal consequences*. Lean supplies a suitable initial checking foundation. Leant supplies implemented candidate-generation and interaction mechanisms with useful distinctions that the new design should preserve and strengthen. The nine reports contribute complementary examples, contracts, and evaluation safeguards.

No report, and no synthesis of their agreement, establishes that the resulting language will be easier to learn or maintain. The central claim remains empirical: making a mathematical move the unit of authorship should reduce representation and assembly work without hiding meaning. A small shared implementation, honest Lean baselines, and carefully chosen failures can test that claim.

This document delivers a comparative design judgment, a static review of selected current implementation paths, and a next-round experimental specification. Its mathematical explanations were reviewed as mathematics; its new pseudocode was not compiled in Lean; the architecture and assembly argument are not a verified compiler metatheory. The completed PDF and its layout checks establish document production quality, not proof-language correctness.

A Report-by-report convergence crosswalk

The entries below identify sections explicitly supporting each shared commitment. The six columns are: **H**, Lean host; **C**, claims/checkpoints; **O**, scoped obligations and method contracts; **F**, fixed meaning and object/statement fidelity; **R**, retained proof and replay; **E**, fair evaluation including shared abstraction costs. All entries are positive source evidence, not scores. More detailed line locators appear in the companion register and reading notes.

Report	H	C	O	F	R	E
Contour	14.1	4.3	9, 11.1	9.1	13.4	15.1–3
Loom	3.1, 12.1	3.2	8.1–4	3.4, 7.4	10.1	13.1–2
MathStep	18.1	3.3	9.1, 11.3	4.2, 14.3	15	19
MPL	16.1	5.2	5, 11.3	12	14.4	17
Mosaic	11.1	4–5	9–10	5.3, 10.4	9.5	13
Motive	4.2, 10	4	6.1–3	4.2, 9.2	9.5	11.1–3
Outline	5.1, 15.1	4.1	7.1, 10	4.2	12.4–5	14.2–3
Reason	16.1	3.2	3.4, 10	11.1	13.3	17
Spine	15	3	7	7.1, 9	12	16
Reports supporting	9/9	9/9	9/9	9/9	9/9	9/9

B Source and artifact map

The synthesis is `docs/synthesis/unified-report.tex` and its rendered `unified-report.pdf`. The companion `source-register.json` records source revisions, file identities, and crosswalk locators. `README.md` describes rebuilding and validation. Reading and implementation-review notes are under `review-notes`. Original report files are listed below relative to `docs/ideas`; their PDF neighbors are linked in the bibliography.

```
contour-proof-language/contour.tex
loom_proof_language/loom.tex
MathStep/MathStep.tex
mathematical_proof_language/article.tex
mosaic_proof_language/mosaic.tex
motive_proof_language_article/motive.tex
outline_proof_language/outline_proof_language.tex
reason_proof_language/reason.tex
Spine_Proof_Language/Spine_Proof_Language.tex
```

Selected local repository inspections are identified below. These are additional observations in this synthesis and do not retroactively change the source provenance of the original reports. All cited implementation behavior is static source evidence unless expressly labeled otherwise.

The later comparison has its own `secondary-source-register.json` and `review-notes/secondary-synthesis.md`. The archive `evidence/secondary-review-source.zip` preserves selected files from the named Claude branch at `70020efd`; its audit numbers remain attributed to that review. `evidence/mathlib-says-source.zip` contains the inspected `says` source and license from Mathlib revision `81a5d257`. These additions leave the nine-report support count and the captured Leant snapshot unchanged.

Source	Inspected mechanism or declaration
ProveIt / BinomialInversion	`sum_Icc_neg_one_pow_choose_mul_choose`, `binomial_inversion`; interval reindexing and additive-group generality.

Source	Inspected mechanism or declaration
ProveIt / AutonomousIteratedDeriv	`AutonomousODE.iteratedDeriv_eq_comp`; neighborhood equality and range-local derivative assumptions.
ProveIt / FloorSqrtSum	`floorSqrtSumClosedForm_cast`, `sum_floor_sqrt_eq_closedForm`; exact division and subtraction guards.
Leant / Main	`goalCandidates`, `suggestTactic`, `probeOnce`, `parseTryThis`, `cmdQed`; candidate selection, response checks, displayed edits, and save behavior.
Leant / Synth.Verification	`Verified`, `verifyCandidateGroups`; callback receipt and lazy quota-aware verification.
Leant / Synth.Fragment	Structural translation, unsupported content, and completeness boundaries.
Leant / Synth.Replay	`planReplay`; reuse versus suffix versus full history replay.
Leant / Backend, tests	Session process/request handling and the `prove-suggest` behavioral transcript.

References

[1] Claude (Anthropic). *Nine Blueprints for a Mathematician's Proof Language over Lean*. 5 September 2026. Brainstorm branch `claude/lean-language-improvements-report-2b93cb`, revision `70020efd267f43131b8ef386ea40577a9e67b7cd`, `docs/unified_report/unified_report.tex` and companion audit artifacts. Archived reviewed sources.

[2] Mathlib contributors. *Mathlib.Tactic.Says*, revision `81a5d257c8e410db227a6665ed08f64fea08e997`, especially lines 90–108. Pinned implementation; Archived source and license.

[3] *Contour: A Human-Scale Proof Language over Lean*. 5 September 2026. Input report PDF.

[4] *Loom: A Contract-Based Language for Human-Scale Formal Proofs*. 5 September 2026. Input report PDF.

[5] *MathStep: A Proof Language of Mathematical Steps*. 5 September 2026. Input report PDF.

[6] *Mathematical Intent, Formal Evidence* (MPL). 5 September 2026. Input report PDF.

[7] *Mosaic: Mathematical Intent, Explicit Obligations, and Kernel-Checked Proof*. 5 September 2026. Input report PDF.

[8] *Motive: Concise Mathematical Reasoning with Kernel-Checked Elaboration*. 5 September 2026. Input report PDF.

[9] *Proofs at the Scale of Ideas: An Intent-Directed, Lean-Checked Proof Language* (Outline). 5 September 2026. Input report PDF.

[10] *Reason: A Proof Language of Mathematical Intent*. 5 September 2026. Input report PDF.

[11] *Spine: A Proof Language for Mathematical Intent*. 5 September 2026. Input report PDF.

[12] Lean team. *Elaboration and Compilation*, Lean Language Reference. `https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/`. Consulted 5 September 2026 Pacific; moving documentation, not a repository toolchain pin.

[13] Lean team. *Validating a Lean Proof*, Lean Language Reference. `https://lean-lang.org/doc/reference/latest/ValidatingProofs/`. Consulted 5 September 2026 Pacific.

[14] Isabelle team. *Programming and Proving in Isabelle/HOL*, chapter on Isar. `https://isabelle.in.tum.de/dist/library/Doc/Prog_Prove/Isar.html`. Consulted 5 September 2026 Pacific.

[15] V. Reshetnikov et al. ProveIt, `Analysis/FabiusFunction/Lean/FabiusFunction/BinomialInversion.lean`. Pinned source, selected lines 43–143.

[16] ProveIt, `Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean`. Pinned source, lines 40–65.

[17] ProveIt, `NumberTheory/IntegerSums/Lean/IntegerSums/FloorSqrtSum.lean`. Pinned source, selected lines 86–163.

[18] V. Reshetnikov et al. Leant, `README.md`. Reviewed working-tree version in the source snapshot; sections on interactive proving, synthesis, behavior, and impossibility. File digest in `source-register.json`.

[19] Leant, `src/Main.hs`. Reviewed working-tree version in the source snapshot; selected suggestion and proof-completion paths, lines 4704–5184 and 5412–5480. This file differs from HEAD.

[20] Leant, `src/Leant/Synth/Verification.hs`. Reviewed working-tree version in the source snapshot; callback receipts and candidate-group scheduler. This file differs from HEAD.

[21] Leant, `src/Leant/Synth/Fragment.hs`. Pinned source; selected capability and structural-translation boundaries.

[22] Leant, `src/Leant/Synth/Replay.hs`. Pinned source; `ReplayPlan` and `planReplay`.

[23] Leant, `src/Leant/Backend.hs`. Pinned source; request and process lifecycle.

[24] Leant, `test/prove-suggest.txt` and its golden output. Pinned test input. Inspected, not rerun for this review.

[25] Leant, `src/Leant/Synth/Engine.hs`. Pinned source; preparation, typed origins, engine limits, and qualified negative outcomes. Selected lines 259–294, 831–840, 1119–1554.

[26] Leant, `src/Leant/Synth/PostVerification.hs`. Pinned source; scoped occurrence handles and checked permutations.

[27] Leant, `docs/candidate-quality.md`. Pinned source; profiles, structural costs, and limits of ranking.

[28] Leant, `docs/length-ranking.md`, `src/Leant/Synth/Length/Selection/Generic.hs`, and `src/Leant/Synth/Length/Integration.hs`. Pinned design/implementation documentation; selected sources also retained in the snapshot.

[29] Leant working tree, `src/Leant/Synth/Behavioral.hs`, `docs/behavioral-synthesis.md`, and `test-unit/Spec.hs`. Uncommitted sources in the review snapshot; parsing, exact proposition checks, verdicts, and stated integration-validation boundary.

[30] Lean community REPL, locally cached source tagged `repl_v1.3.18_lean-toolchain-v4.32.0`, `REPL/Main.lean`, lines 249–323, and `REPL/JSON.lean`, lines 240–256. The protocol source snapshot retains these files and its license; provenance is recorded in the source register.